

Day 3 — Mastery Workshop

Curriculum alignment: Stage 1 — Foundations of Computation · Electricity, States, Bits, and Information

Target time: ~90 minutes (≈20 minutes retrieval, ≈50 minutes practical work, ≈20 minutes explanation)

Can you rebuild the idea without the notes?

Day 1 moved from physical state to bits. Day 2 moved from bit patterns to meaning. Today is different: there is very little new content. The purpose is to discover whether those ideas now belong to you—or still belong to the PDF.

1. The story so far

On Day 1 we started in the physical world:

```
continuous, noisy voltage
    ↓ engineered thresholds
stable logical 0 or 1
    ↓ combine independent choices
2^n possible patterns
```

On Day 2 we added interpretation:

```
bit pattern + encoding/context → meaning
```

Put the two days together:

```
PHYSICS          ABSTRACTION      REPRESENTATION    INTERPRETATION
voltage/charge → logical 0 or 1 → bit pattern → useful meaning
```

That line is the spine of this foundation. Today we test every arrow.

What this foundation made possible

Suppose you want a machine to add two numbers. Saying “build an adder” skips several prerequisites:

1. The numbers need a physical representation.
2. The representation must survive noise and imperfect components.
3. Each possible input pattern must produce a repeatable output pattern.
4. Intermediate results must remain available for the next step.
5. Some mechanism must decide which operation happens next.

Days 1 and 2 solve only the first two items—but without them, the rest cannot even be specified.

This is also how the historical path unfolded. Punched-card tabulators could encode and count facts, but they were not yet general-purpose CPUs. Relay calculators could wire switching paths for arithmetic, but changing the task often meant changing wiring or control media. Electronic switching made operations faster, while stored programs eventually made the sequence of operations itself ordinary encoded information.

The CPU we are heading toward is therefore not one magical invention. It is a stack of solved problems:

```
representation
+ logic
+ arithmetic
+ memory
+ control
+ stored instructions
= a programmable processor
```

Today's mastery test checks that the first layer is firm enough to carry the next one.

2. Why retrieval comes before rereading

Rereading feels smooth because the answer is visible. That feeling can be mistaken for understanding.

Retrieval is less comfortable. You close the notes and try to reconstruct the idea. The missing steps become obvious.

So before reading further, take a blank sheet and answer:

1. Why isn't electricity naturally binary?
2. Why do valid LOW and HIGH values occupy ranges?
3. Why does one additional bit double the number of patterns?
4. What is the difference between a pattern and an encoding?
5. Why does hexadecimal reveal bit structure better than decimal?

Do not worry about polished language. Mark any point where your explanation turns into "because computers work that way." That phrase usually hides the exact bridge we need to rebuild.

3. Challenge one — repair a broken explanation

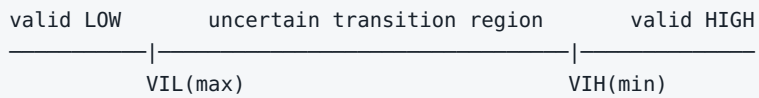
Someone writes:

```
A computer stores zero as exactly 0 volts and one as exactly 3.3 volts. Eight such voltages
make a byte. The byte 01000110 is the letter F.
```

This sounds plausible, but it compresses several layers into one sentence. Repair it.

Problem A: exact voltages

Real electrical values fluctuate. Logic specifications therefore define guaranteed ranges, not perfect points:



The thresholds vary by technology. The principle is stable: broad valid regions create tolerance to noise and component variation.

Problem B: “eight voltages make a byte”

Eight physical signals *can implement* an eight-bit value, but the byte is a logical grouping, not a special electrical object. It could also be stored as charge, magnetism, light, or transmitted over one wire at eight different times.

Problem C: “the byte is F”

The byte pattern is not inherently **F**. Under ASCII it maps to **F**; under unsigned binary it maps to 70; under a bitmap convention it maps to eight pixels.

A repaired explanation might say:

Circuits map broad ranges of physical voltage to logical 0 and 1. Eight logical bits form one of 256 possible byte patterns. An encoding such as ASCII may interpret the pattern 01000110 as the character **F**.

Notice how each noun now belongs to the correct layer.

4. Challenge two — derive instead of quote

Why do five bits produce 32 patterns?

Do not answer only with $2^5 = 32$. Derive it:

0 bits: 1 empty pattern
1 bit: each old pattern gets a 0-version and a 1-version → 2
2 bits: double again → 4
3 bits: double again → 8
4 bits: double again → 16
5 bits: double again → 32

The exponent is not a magic formula. It records repeated independent choices.

Now ask the reverse question:

How many bits are required to distinguish at least 20 possible states?

Four bits are insufficient because they provide only 16 patterns. Five bits provide 32. Some patterns may remain unused; an encoding does not have to use every available pattern.

That last fact explains why many real formats reserve bit patterns for errors, control values, or future expansion.

5. Challenge three — conversion as place value

Consider:

```
101101102
```

Binary to decimal

Write the place values:

```
bits:      1  0  1  1  0  1  1  0
place values: 128 64 32 16  8  4  2  1
```

Add only the active positions:

```
128 + 32 + 16 + 4 + 2 = 182
```

Binary to hexadecimal

Group from the right:

```
1011 0110
B     6
```

Therefore:

```
101101102 = 18210 = 0xB6
```

Now perform these without a calculator:

1. 00101101₂ → decimal and hexadecimal
2. 0xA7 → binary and decimal
3. decimal 99 → binary and hexadecimal

Record the method, not just the answer. If an answer is wrong, identify whether the mistake came from place values, arithmetic, or nibble grouping.

6. Challenge four — prediction on Linux

We will create a file containing one byte:

```
printf '\x41' > /tmp/week1-check.bin
```

Before running anything else, predict each result:

```
xxd -b /tmp/week1-check.bin
od -An -tu1 /tmp/week1-check.bin
cat /tmp/week1-check.bin
objdump -D -b binary -m i386:x86-64 /tmp/week1-check.bin
```

Write four predictions first. Then run the commands.

What you are testing

- `xxd -b` asks for a binary representation.
- `od -tu1` asks for an unsigned decimal byte.
- `cat` sends the byte to a terminal, which interprets it using a character encoding.
- `objdump` asks an x86-64 instruction decoder.

One stored byte can produce four different displays because every tool asks a different question.

Do not memorize the exact `objdump` result. The important prediction is that the decoder will treat `0x41` as machine-code syntax rather than as the character `A`.

7. Run the interactive mastery challenge

The Day 2 HTML Bit Lab is useful for visual exploration. Today's challenge is a small terminal program with randomized conversion input:

```
python3 ./daily/challenges/day-003-mastery.py
```

It tests five objectively checkable items, then asks for the parts software cannot honestly grade: drawing and explanation.

Do not treat `5/5` as proof of mastery. A script can verify an answer string; it cannot determine whether you can rebuild the causal story from voltage to meaning.

8. The mastery checkpoint

The tracker asks:

Explain why the same bit pattern can represent a number, character, instruction, or pixel.

Build the answer in three moves.

Move 1: state what exists

Physical storage contains distinguishable states represented logically as a bit pattern.

Move 2: remove inherent meaning

The pattern by itself does not contain a label saying "number" or "text."

Move 3: supply the interpreter

An encoding and the software or hardware reader decide how to map the pattern to meaning.

Complete answer:

The same pattern can represent different things because storage preserves bits, not semantic labels. An unsigned-number rule, a character encoding, an ISA decoder, or a bitmap format can each map those identical bits to a different meaning. The reader and context change; the bits do not.

Now say it again without using that wording.

9. Explain, draw, observe, build

The tracker's completion standard has four parts.

Explain

Without notes, answer:

- Why is binary physically practical?
- Why do n bits produce 2^n patterns?
- Why does a pattern need an encoding?

Draw

Draw this flow from memory and annotate every arrow:

physical state → logical bit → pattern → interpretation → meaning

Observe

Reproduce the `/tmp/week1-check.bin` experiment from §6 and explain why each tool displays something different.

Build

The conversion checker and interactive Bit Lab already exist. For stronger evidence, change the byte in §6 to one of your choosing and correctly predict all four views before running them.

10. Decide honestly whether this foundation is complete

Use this rubric:

Ready to complete

- You can explain the whole chain without reading.
- You can draw it and explain every arrow.
- You completed the conversions and understand your mistakes.
- You reproduced at least one Linux interpretation experiment.

- You can answer the mastery checkpoint in your own words.

Review required

- You recognize explanations but cannot reconstruct them.
- Binary conversion still depends on guessing.
- “The byte is the letter” still feels identical to “ASCII interprets the byte as the letter.”
- You cannot explain why voltage *ranges* matter.

“Review required” is useful evidence, not failure. It tells us exactly what Day 4 should repair.

Do not fill the tracker’s confidence score merely from reading these pages. Choose it after attempting the challenges.

11. Record your stopping point

Update `./DAILY_PROGRESS.md` with:

- what you completed,
- what remains,
- conversion mistakes,
- questions that appeared,
- whether you can explain the mastery checkpoint,
- and the exact next action.

This is what allows the next session to resume from understanding rather than from a filename or calendar date.

References used selectively

- Charles Petzold, *Code*, Chapters 2–3, 9, and 15 (PDF chapter openings: pages 18, 24, 80, and 199).
- Patt and Patel, *Introduction to Computing Systems*, 2nd ed., Chapter 2, “Bits, Data Types, and Operations,” book pages 21–36.
- Computer History Museum, “The Stored Program,” for the problem of replacing laborious machine setup with instructions held in memory: <https://www.computerhistory.org/revolution/birth-of-the-computer/4/87>

The mastery exercises are original to this course and are designed around the authoritative curriculum tracker.

Next step depends on evidence: if this foundation is stable, Day 4 begins signed-number motivation and two’s complement. If not, Day 4 will target the specific weak link instead of repeating the whole lesson.